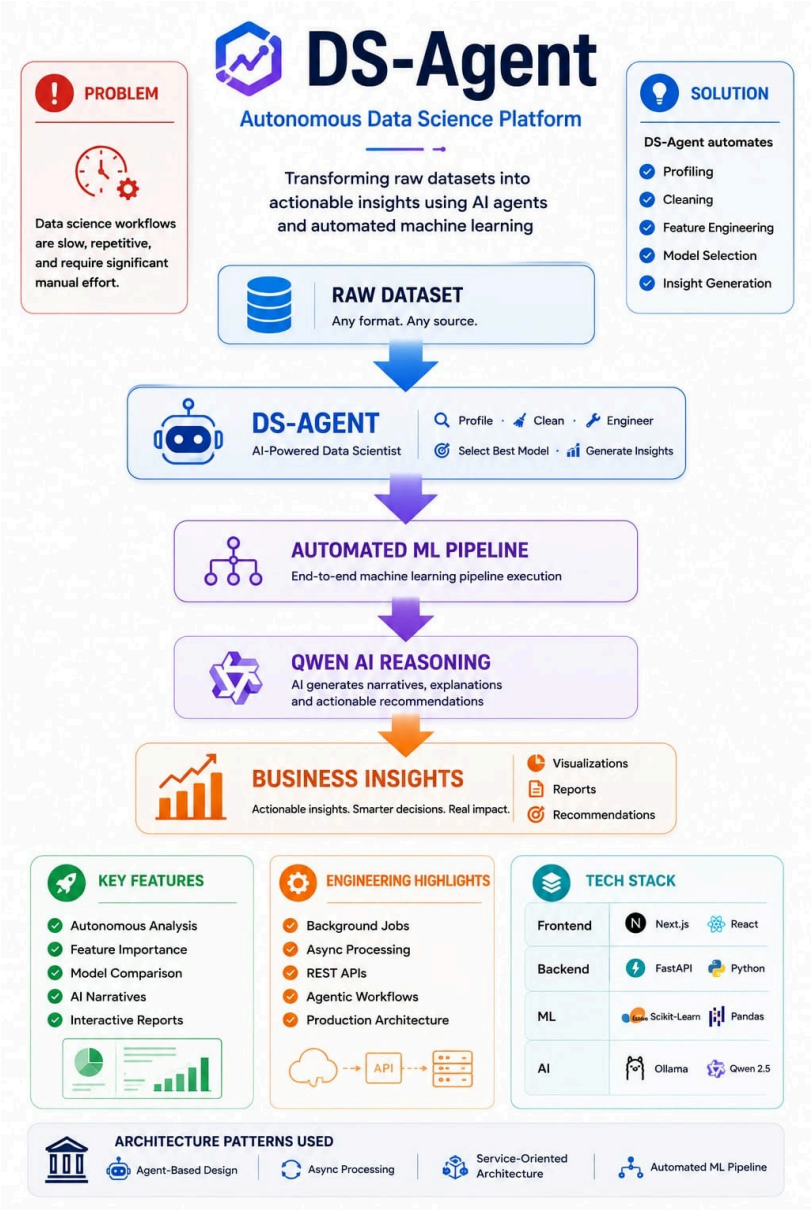
AI-Powered Data Science Agent

DS-Agent is a full-stack autonomous data science platform that transforms a raw CSV into dataset profiling, data cleaning, feature engineering, model training, chart generation, HTML reporting, and AI-generated narrative insights with minimal human intervention. It combines a Next.js frontend, FastAPI backend, classical ML pipelines, and local LLM inference through Ollama and Qwen2.5.

Executive Graphic



This should visually summarize the full value proposition in one frame:

- Upload dataset.
- AI agent analyzes and improves it.
- Models are trained and compared.
- Visualizations and report are generated.
- Narrative explanation appears asynchronously.

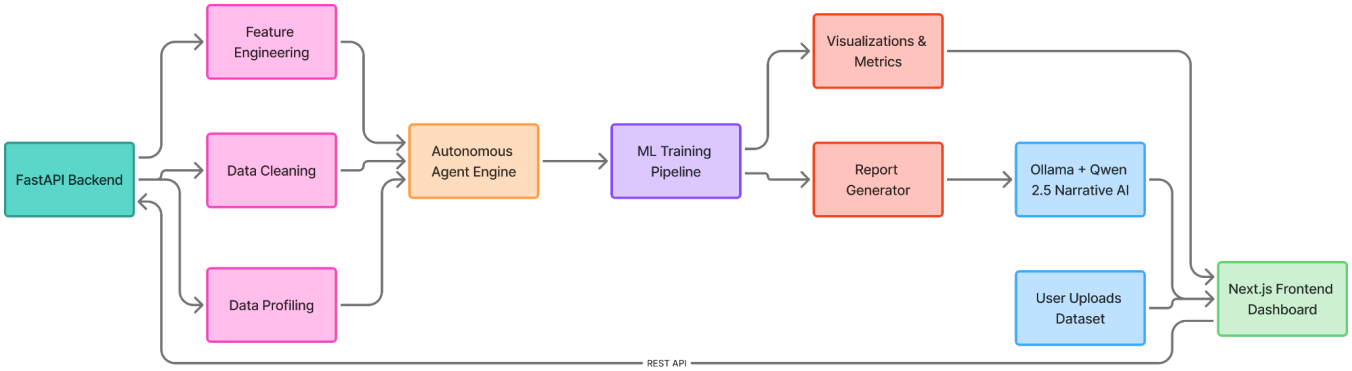
What It Does

- Accepts tabular CSV datasets through a web interface.
- Automatically profiles columns, missing values, uniqueness, and numeric distributions.
- Cleans and normalizes data before modeling.
- Performs controlled feature engineering with safeguards against feature explosion.
- Runs an autonomous improvement loop to evaluate and refine model performance.
- Compares multiple ML models and selects the best performer.
- Generates charts, feature importance, and downloadable HTML reports.
- Produces natural-language executive summaries and recommendations using Qwen2.5 via Ollama.

Architecture

Detailed architecture is available in docs/architecture.md.

- **System Architecture** — shows frontend, backend, pipeline, and LLM boundaries.
- **ML Pipeline** — profiling, cleaning, feature engineering, training, reporting.
- **Async Processing** — background jobs, progress tracking, result retrieval.
- **Agent Decision Loop** — iterative improvement with early stopping.



Why This Project Stands Out

DS-Agent is not just an ML demo; it is a production-style autonomous workflow with real engineering tradeoffs. It solves common failure points such as blocking requests, hidden progress, feature explosion, narrative latency, and unsafe file uploads.

Engineering highlights

- Background-job architecture keeps the UI responsive during long analyses.
- Step-level progress tracking gives users visibility into pipeline execution.
- Cardinality-aware feature engineering prevents runaway feature growth.
- Narrative generation is decoupled from the core ML result path.
- Upload sanitization reduces path traversal risk.

Tech Stack

Layer	Tools
Frontend	Next.js, React, TypeScript, TailwindCSS
Backend	FastAPI, Python, Uvicorn
ML	Pandas, NumPy, Scikit-Learn
Visualization	Matplotlib, Seaborn
LLM	Ollama, Qwen2.5:3B
DevOps	Git, GitHub, Docker-ready design

Demo

- Full demo video: ds-agent-recording.mp4
- Screenshots: ds-agent-ss

Results

The project evolved through several important optimizations:

- Feature count reduced from 10,394 to 29 after cardinality safeguards.
- Training time improved from 363 seconds to 1.58 seconds.
- Blocking narrative generation was converted into an async background task.
- Progress tracker was fixed to reflect real backend step IDs.
- File uploads were sanitized for security.

Future Work

- GPU acceleration for faster local LLM inference.
- Optional cloud LLM providers such as OpenAI, Anthropic, and Gemini.
- Advanced AutoML with hyperparameter search and ensembles.
- Time-series forecasting and deep learning support.
- Multi-agent decomposition for cleaning, feature engineering, model selection, and reporting.
- Direct database connectivity for MySQL, PostgreSQL, and MongoDB.